TEMPORADA XIV

CI NICARAGUA · 2026

DOCUMENTO DE LECTURA

03

// Unidad I · Fundamentos de Java

PROGRAMACIÓN ORIENTADA A OBJETOS

DEL MUNDO REAL AL CÓDIGO

EN ESTA LECTURA

01 ¿Por qué nació la POO?
02 Clases y objetos
03 Pilar 1 · Encapsulamiento
04 Pilar 2 · Herencia
05 Pilar 3 · Polimorfismo
06 Pilar 4 · Abstracción
07 Caso integrador
08 Buenas prácticas
09 Para practicar



ELABORADO POR

MARVIN RAFAEL MEZA PINEDA

Facilitador · Centro Nacional de Innovación y Tecnología Francisco "El Chele" Moreno

Fundamentos de Desarrollo Backend con Java

Octubre 2026

CAPÍTULO 01

1 ¿POR QUÉ NACIÓ LA POO?

> La **programación orientada a objetos (POO)** es una forma de organizar el código alrededor de **objetos**: piezas que reúnen datos y comportamiento, igual que las cosas del mundo real.

En las lecturas anteriores todo tu código vivía dentro de `main`: variables sueltas, condiciones y ciclos. Para un programa de 50 líneas eso funciona. Pero un sistema real (un banco, una tienda en línea, un sistema de matrícula) tiene miles de líneas y decenas de personas trabajando en él. Ahí la forma en que **organizas** el código importa tanto como lo que hace.

Dos paradigmas, dos formas de pensar

Un **paradigma** es una manera de pensar y estructurar un programa. Antes de la POO dominaba el paradigma **estructurado o procedimental**: los datos por un lado y las funciones que los modifican por otro.

ESTRUCTURADO / PROCEDIMENTAL

- Datos y funciones **separados**.
- Cualquier parte del programa puede cambiar cualquier dato: **efectos secundarios globales**.
- Al crecer se vuelve **código espagueti**: todo depende de todo.

ORIENTADO A OBJETOS

- Datos y métodos **juntos** dentro de cada objeto.
- Cada objeto protege sus datos y decide **quién y cómo** los cambia.
- Módulos autosuficientes: **alta mantenibilidad y orden**.

Mira la diferencia con un ejemplo concreto: guardar el saldo de tres clientes de un banco.

Sin objetos

```
String nombre1 = "Ana";  double saldo1 = 500;
String nombre2 = "Luis"; double saldo2 = 120;
String nombre3 = "Rosa"; double saldo3 = 900;

// nada impide un saldo imposible
saldo2 = -300;
```

Con objetos

```
Cuenta c1 = new Cuenta("Ana", 500);
Cuenta c2 = new Cuenta("Luis", 120);
Cuenta c3 = new Cuenta("Rosa", 900);

// la cuenta valida sus propias reglas
c2.retirar(300);   // Fondos insuficientes
```

En la versión sin objetos, con 3 clientes ya tienes 6 variables; con 3,000 clientes es imposible. Además, cualquier línea puede dejar un saldo negativo. En la versión con objetos, cada `Cuenta` guarda **sus propios datos** y trae **sus propias reglas**.

Pensar en objetos

Para diseñar con POO, primero identifica los **objetos** del problema y luego pregúntate dos cosas de cada uno: ¿qué **tiene**? y ¿qué **hace**?

Objeto	Atributos · lo que tiene	Métodos · lo que hace
Celular	marca, batería, saldo	`llamar()` `cargar()` `enviarMensaje()`
Estudiante	nombre, carnet, notas	`inscribirse()` `calcularPromedio()`
Cuenta bancaria	titular, saldo	`depositar()` `retirar()` `consultarSaldo()`
Pedido	productos, total, estado	`agregarProducto()` `pagar()` `cancelar()`

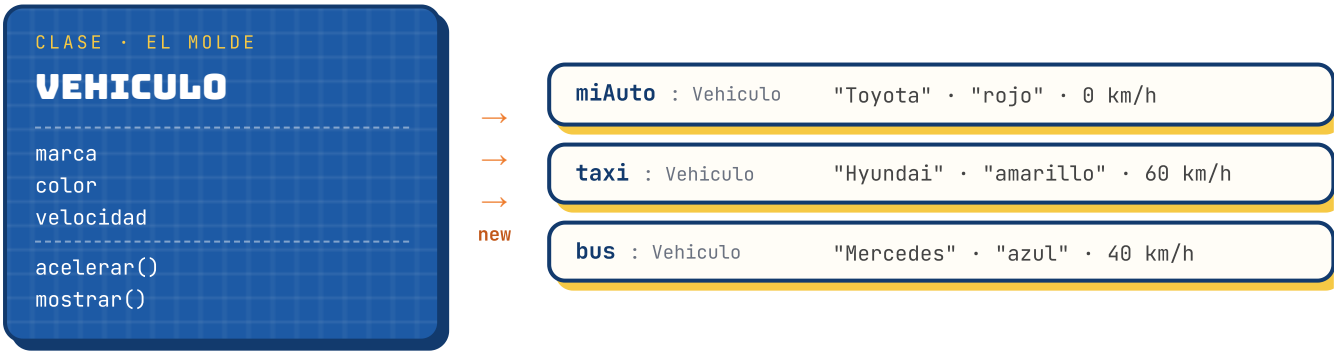
DATO

Java es orientado a objetos desde su diseño en 1995: **todo el código vive dentro de una clase**. Por eso, desde tu primer programa, escribiste `public class HolaMundo` aunque todavía no supieras qué significaba.

CAPÍTULO 02

2 CLASES Y OBJETOS

> Una **clase** es el plano; un **objeto** es la casa construida a partir de ese plano. Con un solo plano puedes construir muchas casas, y cada una tiene su propio color y su propio dueño.



La clase `Vehiculo` no ocupa espacio para datos: solo **describe** qué tendrá cada vehículo. Los objetos (también llamados **instancias**) sí existen en memoria, y cada uno guarda sus propios valores.

Anatomía de una clase

Una clase tiene tres partes principales: atributos, constructor y métodos.

```
Vehiculo.java
```

```java
public class Vehiculo {
    // 1. Atributos: lo que el objeto TIENE
    String marca;
    String color;
    int velocidad;

    // 2. Constructor: cómo se CREA el objeto
    public Vehiculo(String marca, String color) {
        this.marca = marca;
        this.color = color;
        this.velocidad = 0;
    }

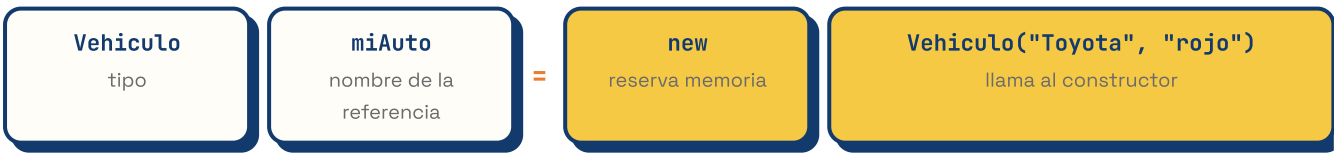
    // 3. Métodos: lo que el objeto HACE
    public void acelerar(int cantidad) {
        velocidad += cantidad;
    }

    public void mostrar() {
        System.out.println(marca + " " + color + " a " + velocidad + " km/h");
    }
}
```

Parte	Qué es
Atributos	Variables declaradas dentro de la clase. Guardan el **estado** de cada objeto.
Constructor	Método especial con el **mismo nombre de la clase** y sin tipo de retorno. Se ejecuta una sola vez, al crear el objeto.
Métodos	Funciones que definen el **comportamiento**. Pueden leer y modificar los atributos del objeto.
`this`	Significa “este objeto”. `this.marca` es el atributo; `marca` a secas es el parámetro.

Crear objetos con new

Para crear un objeto usas la palabra `new` seguida del constructor. Cada pieza de esta línea tiene un trabajo:



Main.java

```java
public class Main {
    public static void main(String[] args) {
        Vehiculo miAuto = new Vehiculo("Toyota", "rojo");
        Vehiculo taxi   = new Vehiculo("Hyundai", "amarillo");

        miAuto.acelerar(30);     // el punto: "objeto, haz esto"
        taxi.acelerar(60);

        miAuto.mostrar();        // Toyota rojo a 30 km/h
        taxi.mostrar();          // Hyundai amarillo a 60 km/h
    }
}
```

Fíjate en que `miAuto.acelerar(30)` no cambia la velocidad de `taxi`: **cada objeto tiene su propia copia de los atributos**. Pon las dos clases en la misma carpeta y ejecuta `Main.java`.

OLVIDAR THIS

Si en el constructor escribes `marca = marca;`, Java asigna el parámetro a sí mismo y el atributo queda en `null`. Cuando el parámetro y el atributo se llaman igual, usa siempre `this.marca = marca;`.

Las variables guardan referencias

Una variable de tipo objeto no guarda el objeto: guarda una **referencia**, la dirección donde vive en memoria. Si copias la variable, copias la dirección, no el objeto:

fragmento

```java
Vehiculo a = new Vehiculo("Kia", "azul");
Vehiculo b = a;          // b apunta al MISMO objeto que a

b.acelerar(50);
a.mostrar();             // Kia azul a 50 km/h
```

Esto explica lo que viste en la Lectura 02: con objetos, `==` compara si dos referencias apuntan al mismo objeto, no si su contenido es igual. Por eso los `String` se comparan con `.equals()`.

Los 4 pilares de un vistazo

Clases y objetos son la base. Sobre ellos se apoyan cuatro ideas, los **pilares de la POO**, que dedicaremos un capítulo cada uno:

1 ENCAPSULAMIENTO

Protege los datos del objeto y solo permite cambiarlos con reglas.

¿Quién puede tocar qué?

2 HERENCIA

Crea clases nuevas a partir de otras y reutiliza su código.

Un Gerente ES UN Empleado.

3 **POLIMORFISMO**

Un mismo mensaje produce respuestas distintas según el objeto.

Mismo método, otra forma.

4 **ABSTRACCIÓN**

Muestra lo esencial y oculta los detalles de implementación.

Qué hace, no cómo lo hace.

CAPÍTULO 03 · PILAR 1

3 ENCAPSULAMIENTO

> **Encapsular** es ocultar los datos internos de un objeto y permitir que se modifiquen **solo a través de métodos** que validan las reglas. Como un cajero automático: no metes la mano en la bóveda, usas la pantalla, y la pantalla revisa cada operación.

El problema: atributos al descubierto

En la clase `Vehiculo` los atributos no tienen protección. Cualquiera puede hacer esto:

fragmento

```
CuentaBancaria cuenta = new CuentaBancaria("Ana", 500);
cuenta.saldo = -5000;       // compila sin errores y nadie lo impide
```

El programa no se cae, pero los datos quedan en un estado imposible. En un banco real, eso es un error grave.

Modificadores de acceso

Java controla quién puede ver cada atributo o método con cuatro niveles de acceso:

Modificador	Misma clase	Mismo paquete	Subclase	Todo el proyecto
`private`	sí	no	no	no
(sin modificador)	sí	sí	no	no
`protected`	sí	sí	sí	no
`public`	sí	sí	sí	sí

La regla práctica: **atributos** `private`**, métodos** `public`. Solo abres lo que otros necesitan usar.

Una clase bien encapsulada

CuentaBancaria.java

```
public class CuentaBancaria {
    private String titular;          // nadie fuera de la clase los ve
    private double saldo;

    public CuentaBancaria(String titular, double saldoInicial) {
        this.titular = titular;
        if (saldoInicial >= 0) {
```

```
            this.saldo = saldoInicial;
        }
    }

    public void depositar(double monto) {
        if (monto > 0) {
            saldo += monto;
        } else {
            System.out.println("El depósito debe ser mayor que 0");
        }
    }

    public boolean retirar(double monto) {
        if (monto > 0 && monto <= saldo) {
            saldo -= monto;
            return true;
        }
        System.out.println("Fondos insuficientes");
        return false;
    }

    public double getSaldo() {        // solo lectura
        return saldo;
    }

    public String getTitular() {
        return titular;
    }
}
```

dentro de main()

```
CuentaBancaria cuenta = new CuentaBancaria("Ana", 500);
cuenta.depositar(200);
cuenta.retirar(1000);                    // Fondos insuficientes
System.out.println(cuenta.getSaldo());   // 700.0

// cuenta.saldo = -5000;   // ERROR de compilación: saldo has private access
```

Getters y setters

Por convención, el método que **lee** un atributo se llama `getAtributo()` (o `isAtributo()` si es `boolean`), y el que lo **modifica** se llama `setAtributo()`. Un buen setter no se limita a asignar: **valida**.

Estudiante.java

```java
public class Estudiante {
    private String nombre;
    private double nota;

    public double getNota() {
        return nota;
    }

    public void setNota(double nota) {
        if (nota < 0 || nota > 100) {
            System.out.println("Nota inválida: debe estar entre 0 y 100");
            return;                  // sale del método sin cambiar nada
        }
        this.nota = nota;
    }

    public boolean isAprobado() {
        return nota >= 70;
    }
}
```

SIN SETTER A PROPÓSITO

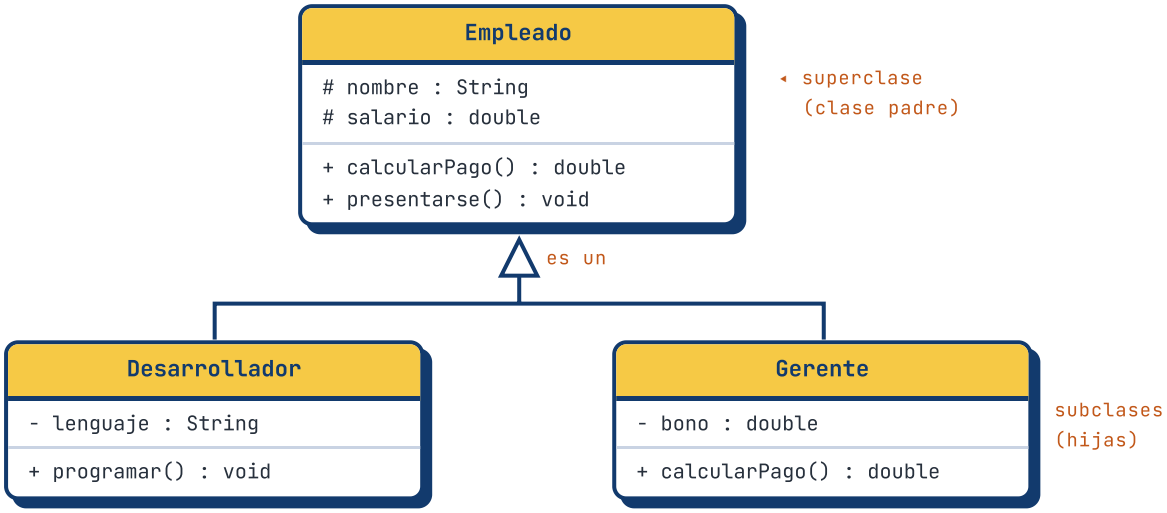
No todo atributo necesita un setter. `CuentaBancaria` no tiene `setSaldo()`: el saldo solo cambia con `depositar()` y `retirar()`, que aplican las reglas del negocio. Ahí está la gracia del encapsulamiento.

Atajo en VS Code: clic derecho dentro de la clase → **Source Action...** → **Generate Getters and Setters**. Úsalo, pero luego decide cuáles realmente necesitas.

CAPÍTULO 04 · PILAR 2

4 HERENCIA

> La **herencia** permite crear una clase nueva a partir de otra que ya existe, heredando sus atributos y métodos. Se usa cuando entre las dos hay una relación **“es un”**: un Desarrollador *es un* Empleado.



Este es un **diagrama de clases UML**, el estándar para dibujar clases. La flecha con punta hueca significa “hereda de”. Los símbolos indican el acceso: `-` private, `#` protected y `+` public.

extends y super

Empleado.java

```java
public class Empleado {
    protected String nombre;         // protected: las hijas pueden usarlo
    protected double salario;

    public Empleado(String nombre, double salario) {
        this.nombre = nombre;
        this.salario = salario;
    }

    public double calcularPago() {
        return salario;
    }

    public void presentarse() {
        System.out.println("Hola, soy " + nombre);
    }
}
```

Desarrollador.java

```java
public class Desarrollador extends Empleado {
    private String lenguaje;         // solo lo agrega lo NUEVO

    public Desarrollador(String nombre, double salario, String lenguaje) {
        super(nombre, salario);      // llama al constructor de Empleado
        this.lenguaje = lenguaje;
    }

    public void programar() {
        System.out.println(nombre + " está programando en " + lenguaje);
    }
}
```

dentro de main()

```java
Desarrollador dev = new Desarrollador("Carla", 1200, "Java");
dev.presentarse();     // Hola, soy Carla              <- heredado de Empleado
dev.programar();       // Carla está programando en Java  <- propio
```

`Desarrollador` no repite `nombre`, `salario` ni `presentarse()`: los hereda. Si mañana corriges `presentarse()` en `Empleado`, todas las subclases reciben el cambio.

Palabra clave	Para qué sirve
`extends`	Indica de qué clase hereda: `class Desarrollador extends Empleado`.
`super(...)`	Llama al constructor del padre. Debe ser la **primera línea** del constructor hijo.
`super.metodo()`	Ejecuta la versión del padre de un método (lo usarás en el capítulo 5).
`protected`	Atributo visible para las subclases, pero no para el resto del proyecto.

DATO

En Java una clase solo puede heredar de **una** clase (herencia simple). Y aunque no escribas `extends`, toda clase hereda de `Object`: de ahí vienen métodos como `toString()` y `equals()`.

¿“Es un” o “tiene un”?

El error más común es usar herencia solo para reutilizar código. Antes de escribir `extends`, lee la frase en voz alta:

Relación	¿Suena bien?	Qué usar
Un Perro **es un** Animal	sí	Herencia: `Perro extends Animal`
Un Gerente **es un** Empleado	sí	Herencia: `Gerente extends Empleado`
Un Auto **es un** Motor	no	Composición: el Auto **tiene un** Motor
Un Pedido **es un** Cliente	no	Composición: el Pedido **tiene un** Cliente

Composición: un objeto como atributo de otro

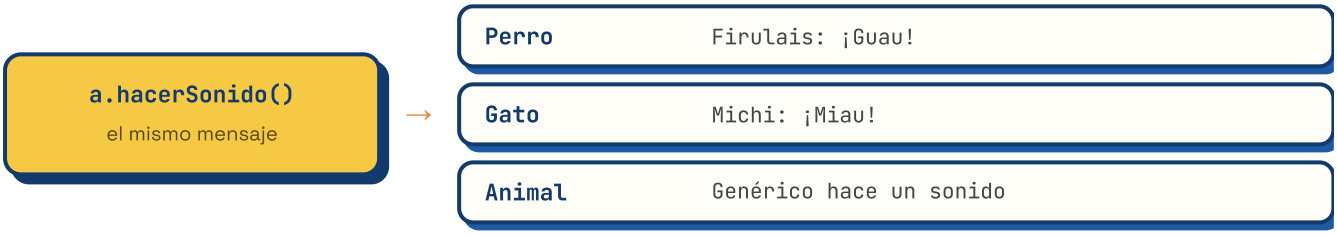
```java
public class Auto {
    private String placa;
    private Motor motor;      // Auto TIENE UN Motor, no ES un Motor

    public void encender() {
        motor.arrancar();     // delega el trabajo al objeto Motor
    }
}
```

CAPÍTULO 05 · PILAR 3

5 POLIMORFISMO

Polimorfismo significa "muchas formas": un mismo mensaje, es decir, llamar al mismo método, produce respuestas distintas según el objeto que lo recibe.



Sobrescritura con @Override

Una subclase puede **reemplazar** un método heredado con su propia versión. Se marca con `@Override`:

Animal.java · Perro.java · Gato.java

```java
public class Animal {
    protected String nombre;

    public Animal(String nombre) { this.nombre = nombre; }

    public void hacerSonido() {
        System.out.println(nombre + " hace un sonido");
    }
}

public class Perro extends Animal {
    public Perro(String nombre) { super(nombre); }

    @Override
    public void hacerSonido() {
        System.out.println(nombre + ": ¡Guau!");
    }
}

public class Gato extends Animal {
    public Gato(String nombre) { super(nombre); }

    @Override
```

```
    public void hacerSonido() {
        System.out.println(nombre + ": ¡Miau!");
    }
}
```

Una variable del padre, un objeto del hijo

Como un Perro *es un* Animal, una variable de tipo `Animal` puede guardar un `Perro`:

fragmento

```
Animal a = new Perro("Firulais");
a.hacerSonido();      // Firulais: ¡Guau!   <- se ejecuta la versión de Perro
```

En `Animal a = new Perro()`	Decide…
El tipo de la variable (`Animal`)	**Qué métodos puedes llamar.** Lo revisa el compilador.
El objeto real (`Perro`)	**Qué versión se ejecuta.** Se decide mientras el programa corre (ligadura dinámica).

Donde el polimorfismo brilla

Su verdadero poder aparece cuando tratas muchos objetos distintos de la misma forma. Un **arreglo** (que veremos a fondo más adelante) guarda varios valores del mismo tipo:

dentro de main()

```
Animal[] animales = {
    new Perro("Firulais"),
    new Gato("Michi"),
    new Animal("Genérico")
};

for (int i = 0; i < animales.length; i++) {
    animales[i].hacerSonido();     // cada objeto responde a su manera
}
```

Salida

```
Firulais: ¡Guau!
Michi: ¡Miau!
Genérico hace un sonido
```

¿POR QUÉ IMPORTA?

Si mañana agregas `class Vaca extends Animal`, el ciclo **no cambia ni una línea.** Sin polimorfismo tendrías un `if` por cada tipo de animal, y tendrías que tocarlo cada vez que aparece uno nuevo.

Un caso de negocio: la planilla

Volvamos a los empleados. El `Gerente` cobra su salario más un bono, así que sobrescribe `calcularPago()` y reutiliza la versión del padre con `super`:

Gerente.java

```java
public class Gerente extends Empleado {
    private double bono;

    public Gerente(String nombre, double salario, double bono) {
        super(nombre, salario);
        this.bono = bono;
    }

    @Override
    public double calcularPago() {
        return super.calcularPago() + bono;    // salario del padre + bono
    }
}
```

dentro de main()

```java
Empleado[] planilla = {
    new Desarrollador("Carla", 1200, "Java"),
    new Gerente("Jorge", 1500, 400),
    new Empleado("Rosa", 900)
};

double total = 0;
for (int i = 0; i < planilla.length; i++) {
    total += planilla[i].calcularPago();      // 1200 + 1900 + 900
}
System.out.println("Planilla total: C$" + total);   // Planilla total: C$4000.0
```

Sobrecarga: mismo nombre, otros parámetros

La **sobrecarga** es otra forma de polimorfismo: varios métodos con el **mismo nombre** en la misma clase, pero con **distintos parámetros**. Java elige cuál usar según los argumentos:

Calculadora.java + uso

```java
public class Calculadora {
    public int sumar(int a, int b)           { return a + b; }
    public double sumar(double a, double b)  { return a + b; }
    public int sumar(int a, int b, int c)    { return a + b + c; }
}

Calculadora calc = new Calculadora();
calc.sumar(2, 3);          // 5    -> usa (int, int)
calc.sumar(2.5, 1.5);      // 4.0  -> usa (double, double)
calc.sumar(1, 2, 3);       // 6    -> usa (int, int, int)
```

Ya la usabas sin saberlo: `System.out.println()` tiene una versión para `int`, otra para `double`, otra para `String`…

	Sobrescritura (`@Override`)	Sobrecarga
Dónde	En una subclase, sobre un método heredado	En la misma clase
Parámetros	Exactamente iguales al del padre	Distintos en número o tipo
Se decide	Al ejecutar (dinámico)	Al compilar (estático)
Ejemplo	`Perro.hacerSonido()`	`sumar(int, int)` y `sumar(double, double)`

USA SIEMPRE @OVERRIDE

Si te equivocas al escribir el nombre, por ejemplo `hacersonido()`, sin `@Override` Java crea un método nuevo en silencio y tu versión nunca se ejecuta. Con `@Override`, el compilador te avisa del error.

CAPÍTULO 06 · PILAR 4

6 ABSTRACCIÓN

> **Abstraer** es quedarse con lo esencial y esconder los detalles. Para manejar un carro usas el volante y los pedales (**qué hace**) sin necesidad de saber cómo funciona el motor por dentro (**cómo lo hace**).

LO QUE VES · QUÉ HACE

La parte pública: lo que cualquiera puede usar.

`volante` `acelerador` `freno` `palanca de cambios`

▼ ABSTRACCIÓN: UNA LÍNEA QUE NO NECESITAS CRUZAR ▼

LO QUE SE OCULTA · CÓMO LO HACE

La implementación: puede cambiar sin que el conductor lo note.

`inyección de combustible` `pistones` `transmisión` `frenos ABS`

En Java, la abstracción se construye con dos herramientas: las **clases abstractas** y las **interfaces**.

Clases abstractas

Una clase `abstract` es una clase incompleta a propósito: **no se puede instanciar** y puede tener **métodos abstractos**, sin cuerpo, que cada hija está obligada a implementar.

Figura.java

```java
public abstract class Figura {
    protected String nombre;

    public Figura(String nombre) {
        this.nombre = nombre;
    }

    public abstract double calcularArea();     // sin cuerpo: cada hija decide

    public void describir() {                   // método normal, se hereda
        System.out.println(nombre + " con área " + calcularArea());
    }
}
```

Circulo.java · Rectangulo.java

```java
public class Circulo extends Figura {
    private double radio;

    public Circulo(double radio) {
        super("Círculo");
        this.radio = radio;
    }

    @Override
    public double calcularArea() {
        return Math.PI * Math.pow(radio, 2);
    }
}

public class Rectangulo extends Figura {
    private double base;
    private double altura;

    public Rectangulo(double base, double altura) {
        super("Rectángulo");
        this.base = base;
        this.altura = altura;
    }

    @Override
    public double calcularArea() {
        return base * altura;
    }
}
```

```
dentro de main()
// Figura f = new Figura("algo");   // ERROR: Figura is abstract; cannot be instantiated

Figura c = new Circulo(2);
Figura r = new Rectangulo(4, 3);
c.describir();                      // Círculo con área 12.566370614359172
r.describir();                      // Rectángulo con área 12.0
```

`describir()` está escrito una sola vez en `Figura`, pero llama a `calcularArea()`, que cada figura resuelve a su manera. Abstracción y polimorfismo trabajando juntos.

Interfaces: un contrato

Una `interface` va un paso más allá: solo declara **qué** métodos debe tener una clase, sin decir nada del **cómo**. Es un contrato. La clase que lo firma usa `implements`:

MetodoPago.java

```
public interface MetodoPago {
    boolean pagar(double monto);      // public y abstract por defecto
}
```

PagoEfectivo.java · PagoTarjeta.java

```
public class PagoEfectivo implements MetodoPago {
    @Override
    public boolean pagar(double monto) {
        System.out.println("Pago en efectivo: C$" + monto);
        return true;
    }
}

public class PagoTarjeta implements MetodoPago {
    private String ultimosDigitos;

    public PagoTarjeta(String ultimosDigitos) {
        this.ultimosDigitos = ultimosDigitos;
    }

    @Override
    public boolean pagar(double monto) {
        System.out.println("Cobrando C$" + monto + " a la tarjeta ****" + ultimosDigitos);
        return true;
    }
}
```

```
Caja.java + uso
public class Caja {
    public void cobrar(MetodoPago metodo, double total) {
        if (metodo.pagar(total)) {          // no sabe ni le importa cuál es
            System.out.println("Venta registrada");
        }
    }
}

Caja caja = new Caja();
caja.cobrar(new PagoEfectivo(), 350);
caja.cobrar(new PagoTarjeta("4821"), 1200);
```

`Caja` depende del contrato `MetodoPago`, no de una clase concreta. Mañana puedes agregar `PagoTransferencia` sin tocar `Caja`.

Clase abstracta o interfaz

Concepto	Clase abstracta	Interfaz
¿Se puede instanciar?	No	No
Atributos	Sí, de cualquier tipo (estado)	Solo constantes
Métodos	Abstractos y con implementación	Abstractos (y `default` desde Java 8)
Constructor	Sí	No
Cuántas por clase	`extends` solo una	`implements` varias
Úsala cuando…	Las clases comparten código y estado (“es un”)	Las clases comparten un comportamiento sin ser parientes (“puede hacer”)

LO VERÁS EN LA UNIDAD II

En Spring Boot trabajarás con interfaces todo el tiempo. Por ejemplo, un repositorio de base de datos es una interfaz que tú declaras y que Spring implementa por ti.

7

CAPÍTULO 07

CASO INTEGRADOR

> Un sistema de notificaciones que envía avisos por correo y por SMS. En pocas líneas aparecen **los cuatro pilares** trabajando juntos.

```
Notificacion.java · EmailNotificacion.java · SmsNotificacion.java
// 1. ABSTRACCIÓN: el contrato, sin detalles de implementación
public interface Notificacion {
    void enviar(String mensaje);
}
```

```java
// 2. HERENCIA E IMPLEMENTACIÓN: la clase firma el contrato
public class EmailNotificacion implements Notificacion {

    // 3. ENCAPSULAMIENTO: estado interno protegido
    private String destinatario;

    public EmailNotificacion(String destinatario) {
        this.destinatario = destinatario;
    }

    // 4. POLIMORFISMO: su propia forma de cumplir enviar()
    @Override
    public void enviar(String mensaje) {
        System.out.println("Email a " + destinatario + ": " + mensaje);
    }
}

public class SmsNotificacion implements Notificacion {
    private String telefono;

    public SmsNotificacion(String telefono) {
        this.telefono = telefono;
    }

    @Override
    public void enviar(String mensaje) {
        System.out.println("SMS al " + telefono + ": " + mensaje);
    }
}
```

1 **Abstracción.** `Notificacion` dice *qué* se puede hacer (enviar), sin decir cómo.

2 **Herencia / implementación.** Cada canal implementa la interfaz y queda obligado a cumplirla.

3 **Encapsulamiento.** `destinatario` y `telefono` son `private`: nadie los cambia desde fuera.

4 **Polimorfismo.** El mismo `enviar()` se comporta distinto en cada clase.

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Notificacion[] canales = {
            new EmailNotificacion("ana@correo.com"),
            new SmsNotificacion("8888-1234"),
            new EmailNotificacion("luis@correo.com")
        };

        for (int i = 0; i < canales.length; i++) {
            canales[i].enviar("Tu pedido fue despachado");
        }
    }
}
```

```
Salida
Email a ana@correo.com: Tu pedido fue despachado
SMS al 8888-1234: Tu pedido fue despachado
Email a luis@correo.com: Tu pedido fue despachado
```

¿Y SI MAÑANA PIDEN WHATSAPP?

Creas `WhatsAppNotificacion implements Notificacion` y la agregas al arreglo. El ciclo de `Main` y las otras clases **no cambian**. Ese es el objetivo de la POO: un sistema que crece agregando piezas, no reescribiendo las que ya funcionan.

8 CAPÍTULO 08 BUENAS PRÁCTICAS

> Conocer los pilares no basta: también hay que saber cuándo **no** usarlos. Estas son las prácticas que separan un código mantenible de uno que nadie quiere tocar.

✓ BUENAS PRÁCTICAS

- Composición sobre herencia.
- Alta cohesión y bajo acoplamiento.
- Inmutabilidad con `record`.
- Nombres que explican: clases con sustantivos, métodos con verbos.

✗ ERRORES COMUNES

- Clases Dios (*God Objects*).
- Cadenas de herencia profundas.
- Getters y setters “ciegos”.
- Atributos `public` por comodidad.

Composición sobre herencia

Si dudas entre heredar o tener un objeto como atributo, elige la composición. La herencia une a padre e hija para siempre: cualquier cambio en el padre afecta a todas sus hijas. Usa `extends` solo cuando el “es un” sea evidente, y evita jerarquías de más de dos o tres niveles.

Alta cohesión y bajo acoplamiento

- **Cohesión:** cada clase hace **una sola cosa** y la hace bien. Una clase `Factura` que además envía correos y guarda en la base de datos es una “clase Dios”: difícil de entender y de probar.
- **Acoplamiento:** cuánto depende una clase de los detalles de otra. `Caja` tiene bajo acoplamiento porque solo conoce la interfaz `MetodoPago`.

Getters y setters ciegos

```
X encapsulamiento de mentira
private double saldo;

public void setSaldo(double saldo) {
    this.saldo = saldo;   // acepta -5000
}
```

```
✓ operaciones con reglas
private double saldo;

public void depositar(double monto) {
    if (monto > 0) saldo += monto;
}
```

Un `private` con un setter que acepta cualquier valor es lo mismo que un `public`. Expón **operaciones del negocio**, no atributos.

Inmutabilidad con records

Desde Java 16, un `record` crea en una línea una clase cuyos datos **no pueden cambiar** después de crearse. Es ideal para datos que solo se transportan, como un producto de un catálogo:

```
Producto.java + uso
public record Producto(String nombre, double precio) { }

Producto p = new Producto("Teclado", 25.5);
System.out.println(p.nombre());    // Teclado
System.out.println(p);             // Producto[nombre=Teclado, precio=25.5]
```

Java genera por ti el constructor, los atributos `private final`, los métodos de lectura (`nombre()`, `precio()`), `equals()` y `toString()`. Lo usarás mucho en las APIs de la Unidad II.

CAPÍTULO 09

9 PARA PRACTICAR

> Resuelve estos ejercicios en tu carpeta `clase-03`. Crea un archivo `.java` por clase y súbelos a tu repositorio.

01 **Tu primera clase** básico

Crea la clase `Libro` con `titulo`, `autor` y `paginas`, un constructor y un método `mostrarInfo()`. En `main`, crea tres libros e imprímelos.

02 **Termostato seguro** básico

Clase `Termostato` con `temperatura` privada. Su setter solo acepta valores entre 16 y 30 °C. Agrega `subir()` y `bajar()`, que cambian un grado sin salirse del rango.

03 **Cuenta con límite** intermedio

Amplía `CuentaBancaria`: no permite retiros mayores de C$5,000 por operación y lleva un contador privado de transacciones con su getter.

04 Flota de vehículos `intermedio`

Crea `Moto` y `Camion` que hereden de `Vehiculo` usando `super`. `Camion` tiene `capacidadToneladas`. Sobrescribe `mostrar()` en ambas.

05 Más figuras `intermedio`

Agrega `Triangulo` a la jerarquía de `Figura`. Guarda un círculo, un rectángulo y un triángulo en un arreglo e imprime el área total.

06 Sistema de pagos `reto`

Con la interfaz `MetodoPago`, agrega `PagoTransferencia`. Haz que `PagoTarjeta` tenga un límite de crédito y devuelva `false` si el monto lo supera. Prueba `Caja` con los tres métodos.

Resumen del documento

Concepto	Palabras clave	Recuerda
Clase y objeto	`class` `new` `this`	La clase es el plano; el objeto, la casa.
Encapsulamiento	`private` `get` `set`	Datos privados; se cambian solo con reglas.
Herencia	`extends` `super` `protected`	Solo si “es un”. Una sola superclase.
Polimorfismo	`@Override`	El objeto real decide qué versión se ejecuta.
Abstracción	`abstract` `interface` `implements`	Define el qué; las clases concretas, el cómo.

Antes de la próxima clase

- Poder explicar con tus palabras la diferencia entre clase y objeto.
- Haber escrito al menos una clase con atributos `private`, constructor y métodos.
- Identificar en el caso integrador dónde aparece cada uno de los cuatro pilares.

Referencias

Oracle. *The Java™ Tutorials: Object-Oriented Programming Concepts.* docs.oracle.com/javase/tutorial/java/concepts

Oracle. *The Java™ Tutorials: Classes and Objects.* docs.oracle.com/javase/tutorial/java/javaOO

Oracle. *The Java™ Tutorials: Interfaces and Inheritance.* docs.oracle.com/javase/tutorial/java/landl

OpenJDK. *JEP 395: Records.* openjdk.org/jeps/395



MARVIN RAFAEL MEZA PINEDA

Facilitador · Fundamentos de Desarrollo Backend con Java

Centro Nacional de Innovación y Tecnología Francisco “El Chele” Moreno · Temporada XIV · 2026